



Computer Vision Group Project

ARI3129

Generative AI Usage Journal

Daniel Pace, Amy Spiteri, Ellyn Rose Debrincat, Joachim Grech

January 31, 2026

Contents

1	Introduction	1
2	Ethical Considerations	1
3	Methodology	1
4	Prompts and Responses	2
5	Improvements, Errors and Contributions	5
6	Reflection	6
7	References and List of Resources Used	i

1 Introduction

Generative AI tools were used in this assignment. The models used include:

- Gemini 3 Pro: Chosen for its advanced thinking capabilities and large context window, making it suitable for complex problem-solving tasks. Also selected thanks to the free subscription provided by the University.
- Claude Sonnet 4.5: Selected for its high performance in coding tasks and problem-solving. The more formal tone it tends to use was also found to be beneficial for academic purposes.
- ChatGPT 5.2: Selected for its versatility in generating diverse solutions and quick troubleshooting.
- Antigravity: Utilized as an agentic AI assistant to streamline the development process, automate repetitive tasks, and perform precise error detection across the codebase and documentation.
- NotebookLM: Chosen for its ability to generate information from specific sources, summarize complex materials, and help structure research findings in a clear, organized way.

2 Ethical Considerations

The integration of generative AI tools into academic research raises several ethical considerations that warrant careful examination and mitigation strategies.

Data Bias and Model Limitations

Generative AI models reflect biases inherent in their training data, which can manifest in code suggestions favouring certain programming paradigms or in technical explanations using biased terminology. To mitigate these concerns, we adopted a critical evaluation approach where all AI-generated content was reviewed and verified against authoritative sources such as official documentation and peer-reviewed papers. AI outputs were treated as starting points for investigation rather than definitive answers.

Privacy and Data Security

Privacy considerations were paramount in our AI usage. We ensured that no sensitive information from our dataset was shared with AI platforms. All interactions were limited to general technical queries and code debugging that did not expose proprietary project details. We remained mindful of data retention policies, recognizing that prompts may be stored and used for model training.

3 Methodology

The use of generative AI in our project can be classified into the following two main categories:

Automatic, in-line code suggestions

In Visual Studio Code, the copilot extension was used to provide real-time code suggestions and completions. This tool was primarily employed during coding sessions to enhance productivity and reduce syntax errors. The AI model provided context-aware suggestions based on the current codebase, which were then reviewed and integrated by the team members. The free student subscription provided by GitHub was utilized for this purpose, giving us access to premium models and features.

Prompt-Response interactions for specific tasks

For other tasks, such as debugging certain issues, or generating documentation outlines, we used web-based interfaces for Gemini 3 Pro, Claude Sonnet 4.5, and ChatGPT 5.2. The process involved formulating specific prompts related to the task at hand, submitting them to the AI model, and then evaluating the generated responses. The outputs were critically assessed for accuracy and relevance before being incorporated into our project workflow.

4 Prompts and Responses

The following are examples of noteworthy prompts and their outputs that highlight how the tools were used in this project.

Creating to-do lists for project management

Before starting the project, a checklist was created to outline the steps needed to complete the assignment, as shown in Figure 1. This checklist allowed us to break down the project into manageable tasks and ensured that we stayed on track throughout the development process.

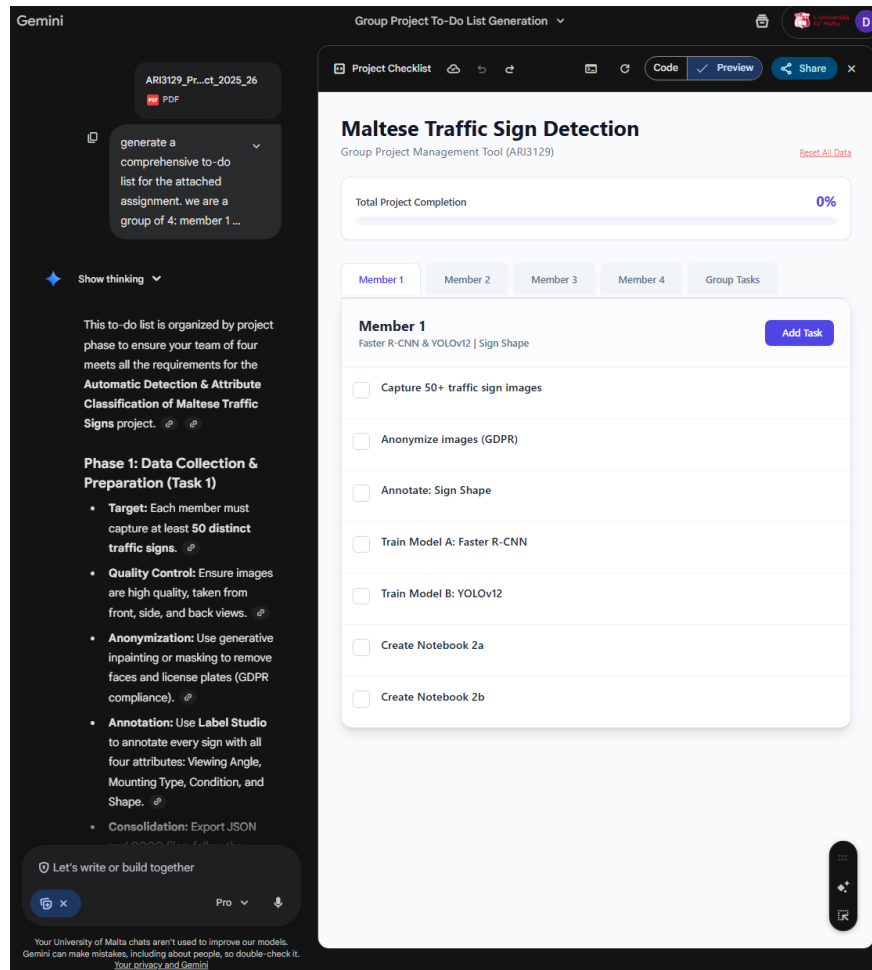


Figure 1: Prompt: Creating a project to-do list

Notably, the checklist html generated by the AI model was fully functional and would remember any modifications made to it, allowing us to track our own progress effectively. Some limitations were observed,

such as the inability to automatically sync the progress of the other members, as well as the contents of the checklist itself not being entirely accurate. The first issue was ignored as it was not worth the effort to implement a more complex solution, while the second issue was resolved by manually editing the checklist to better reflect our project requirements.

The generation of the checklist in Figure 1 was done using 2 prompts, one to generate the initial list of tasks, and another to convert it into an interactive html format.

Generating report outlines

These 2 prompts (Figure 2) illustrate how generative AI was used to structure the report, in accordance with the provided guidelines.



Figure 2: Prompts used to generate report structure

After the initial outputs were generated, we modified the structure slightly to better fit our specific project needs. The AI-generated outlines provided a solid foundation, which we then customized to ensure clarity and coherence in our report.

Debugging and fixing errors

Generative AI tools were also used to help with understanding and debugging errors in our code in an efficient way, as can be seen in Figure 3. Using these tools for help in fixing errors was useful as they gave us a way to quickly understand the main issue behind what was causing the problems, as well as suggesting

ways on how to fix them. Using Generative AI as a coding assistant in this way helped save a lot of time that otherwise would be spent trying to debug confusing errors.

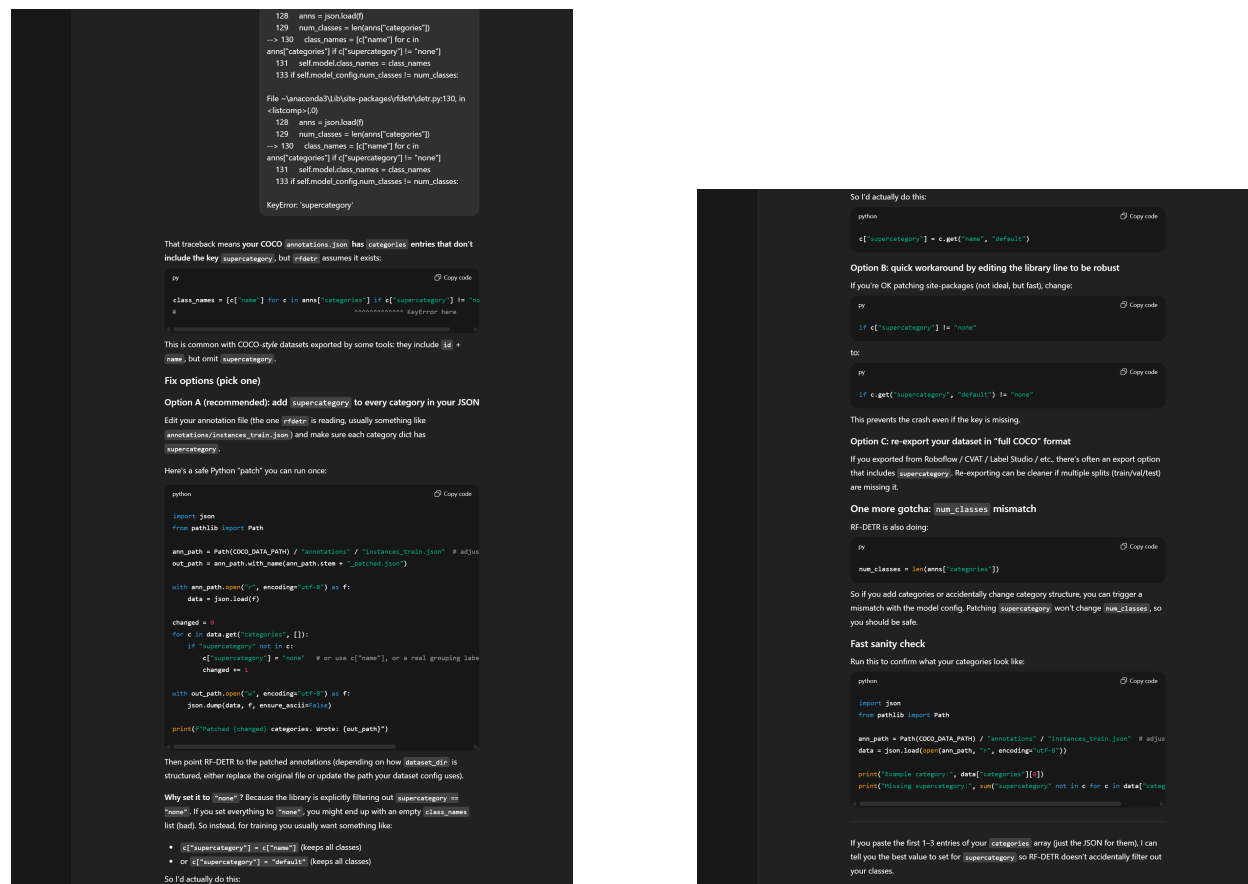


Figure 3: Prompt: Fixing an error

Although Generative AI was helpful in many situations, sometimes it also did not provide helpful solutions to fix the errors that we were encountering. In these situations, it resulted in wasting more time by interacting with the tool without leading to any progress. This highlighted the importance of validating any code suggestions that were given as outputs, as they were not always useful or relevant.

Code suggestions

Throughout the project, AI tools were used to help the development process by providing code suggestions. These suggestions were used for help in certain aspects such as improving the structure and readability of our code, as well as for help with offering guidance on how to implement specific features when we were unsure how to proceed. For example, Figure 4 shows how ChatGPT was utilized to provide a suggestion on how to implement a specific task.

Not all code suggestions given by the AI tools were useful, which is why the output was not just copied but was carefully reviewed and tested before being integrated to ensure that it does what is intended, and that we understood what the code was actually doing.

Literature Review help

NotebookLM was used as a tool to help with the process of writing the Background for the report. It was used to provide summaries of the core concepts in the papers relevant to the object detectors we were researching, helping to gain a better understanding of the content in the paper, as shown in Figure 5. This

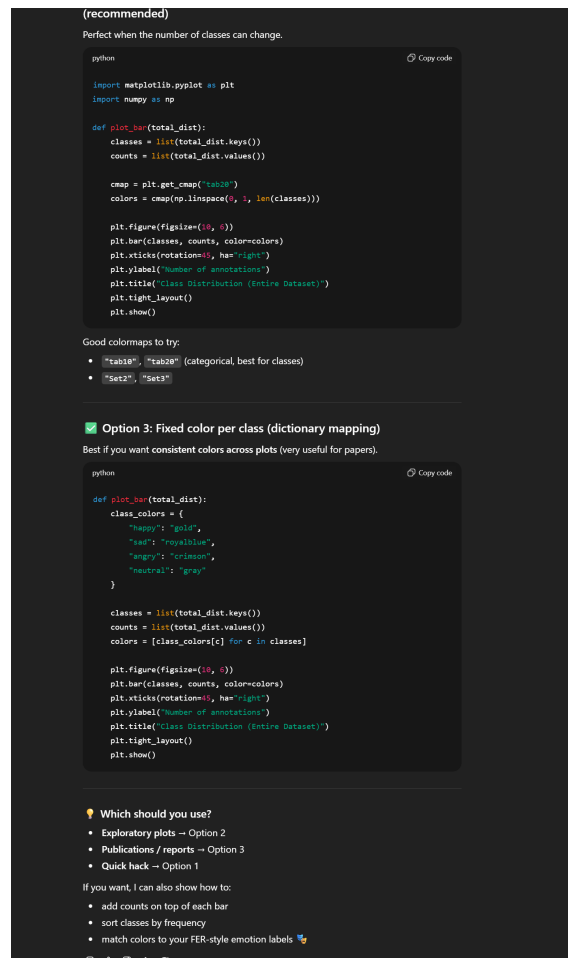
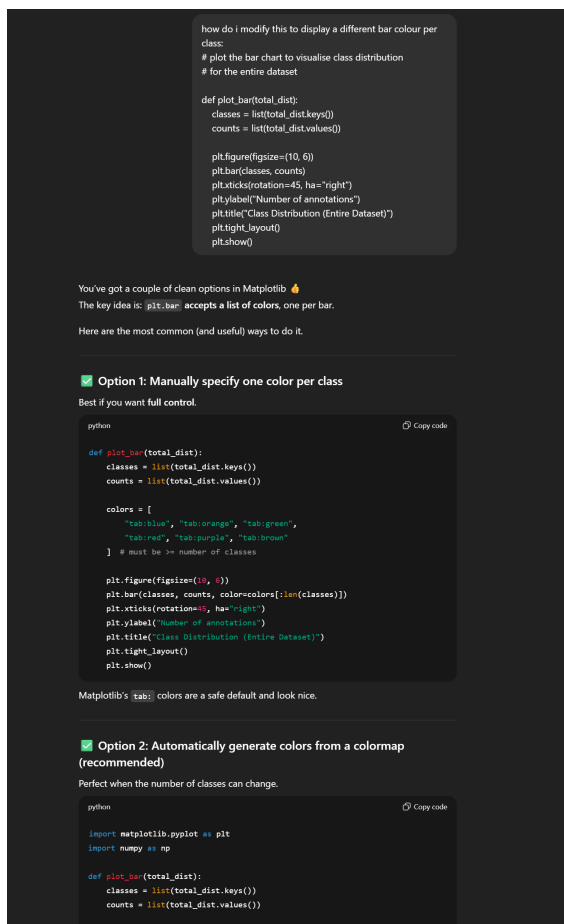


Figure 4: Prompt: Asking for code suggestions

was useful as it made new concepts simpler to understand by providing concise points of the main content in the papers.

After going through the points and explanations given by NotebookLM, the output was verified by reading through the relevant parts of the actual paper itself. This ensured we gained an accurate understanding of the actual content

5 Improvements, Errors and Contributions

Generative AI tools significantly enhanced our efficiency in most aspects of this project. The ability to quickly suggest relevant code snippets, debug issues, and generate documentation outlines allowed us to focus more on higher-level problem-solving and less on routine tasks. However, it was crucial to maintain a critical eye on the AI-generated content, as not all suggestions were accurate or contextually appropriate. This experience has reinforced the importance of human oversight in AI-assisted workflows. Overall, the integration of generative AI has positively influenced our approach to academic projects, highlighting both its potential and limitations.

In particular, generative AI provided assistance in finding relevant library documentation and usage examples, deepening our understanding of the tools we used. It also helped in brainstorming solutions for complex problems, offering alternative perspectives that we might not have considered otherwise. From our experience, the smaller, free versions of these models struggled with understanding the code, as the structure of Jupyter notebooks often either ‘confuses’ the model or perhaps it does not fit in the context

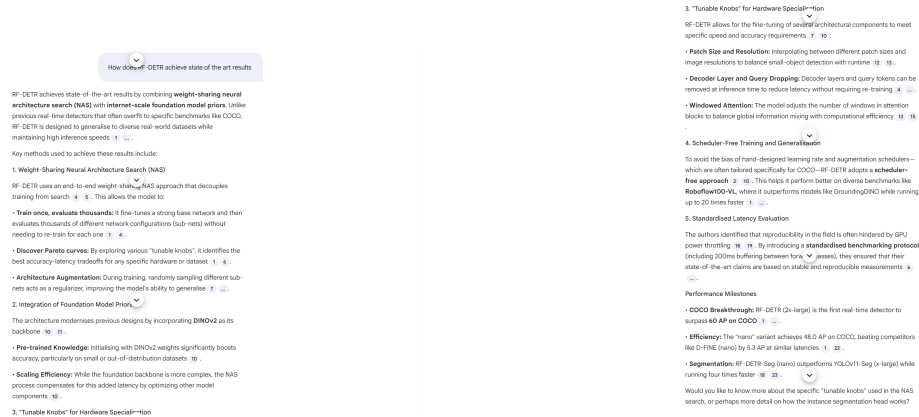


Figure 5: Prompt: Summarizing a paper

window properly. This led to some frustrating experiences where the model would provide irrelevant or incorrect suggestions. However, when using the more advanced models with larger context windows, we found that they were better able to understand the code structure and provide more accurate suggestions.

Aside from code-related tasks, generative AI also contributed to the formulation of the documentation by providing structured outlines and suggestions for content organization. This was helpful in ensuring that the report was well structured, as well as aiding us in splitting up the work among team members, ensuring more-or-less equal contributions.

6 Reflection

Generative AI has become a widely adopted tool in various fields, including academic research and project development. Thus, as students, we believe that it is necessary to make use of these tools in order to better understand their capabilities and limitations.

From a personal perspective, over the 3 years of this degree, we have seen a significant evolution in the capabilities of generative AI tools. Initially, these tools were limited in their ability to understand context and provide relevant suggestions, and would most certainly have struggled to assist with this project. However, advancements in the relevant fields have resulted in more sophisticated models that can comprehend complex code structures and provide meaningful assistance. We’ve realized that, despite the significant improvements in the tools since we first encountered them, they are still able to make mistakes, and thus our trust in them has not increased significantly. We have learned to approach AI-generated content with a critical mindset, verifying its accuracy and relevance before integration.

7 References and List of Resources Used

- [1] Google, “Gemini 3 Pro,” Nov. 2025. [Online]. Available: <https://gemini.google.com/>
- [2] Anthropic, “Claude Sonnet 4.5,” Sep. 2025. [Online]. Available: <https://claude.ai/>
- [3] OpenAI, “ChatGPT (GPT-5.2),” Dec. 2025. [Online]. Available: <https://chatgpt.com/>
- [4] Google DeepMind, “Antigravity: Advanced Agentic Coding Assistant,” 2026. [Online]. Available: <https://antigravity.google/>
- [5] Google, “NotebookLM,” *Google Labs*, 2023. [Online]. Available: <https://notebooklm.google.com/>